

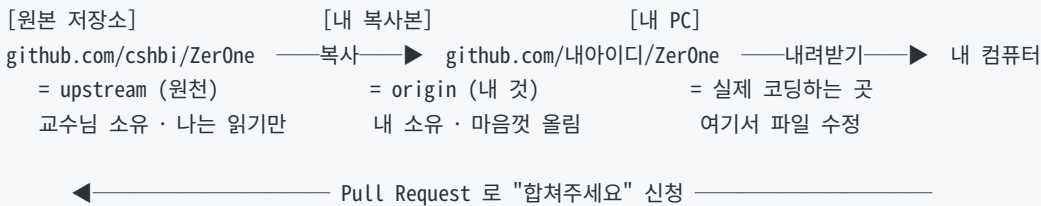
GitHub 협업 실습 가이드

Fork(복사본) → 작업 → Pull Request(합치기 신청) 따라하기

학생용 · 처음 하는 사람 기준

이 방식은 한 문장이면 끝난다. ① 원본을 내 계정으로 복사하고 → ② 내 복사본에서 작업하고 → ③ "원본에 합쳐주세요"라고 신청한다. 원본은 교수님 것이라 내가 직접 못 고치고, 신청(PR)을 교수님이 승인해야 반영된다. 그래서 원본이 안전하다.

★ 먼저 그림부터 — 저장소가 3군데에 있다



단어	뜻 (딱 이것만 기억)
Fork	원본을 내 계정으로 복사하는 것 (웹 버튼 한 번)
origin	내 복사본. 내가 자유롭게 올릴 수 있는 곳
upstream	원본(교수님 것). 최선을 받아오고, 여기로 PR을 보냄
Pull Request (PR)	"내 작업을 원본에 합쳐주세요"라는 신청서

1부 최초 1회만 하는 준비 (셋업)

아래 3단계는 처음 한 번만 하면 된다. 다음부터는 2부만 반복한다.

① 복사본 만들기 — 웹 버튼 (명령어 아님)

브라우저로 원본에 접속 → 우측 상단 **Fork** 버튼 클릭 → 내 계정에 복사본 생성.

```
# 접속할 원본 주소
https://github.com/cshbi/ZerOne
# Fork 버튼 클릭 → https://github.com/내아이디/ZerOne 가 생김
```

② 내 복사본을 PC로 내려받기 (clone)

```
git clone https://github.com/내아이디/ZerOne.git
cd ZerOne
```

제일 많이 하는 실수! 반드시 **내 아이디가 들어간 복사본 주소**를 clone해야 한다. 원본(`cshbi/...`)을 clone하면 나중에 올릴 때 권한 없음으로 막힌다. → 내 복사본 페이지의 초록색 **Code** 버튼에서 주소를 복사해 쓰면 안전.

③ 원본(upstream) 연결 추가

지금은 내 복사본(origin)만 연결돼 있다. 원본 최선을 받아오려면 원본도 등록해 둔다.

```
git remote add upstream https://github.com/cshbi/Zer0ne.git
git remote -v          # 확인
```

아래처럼 두 개가 나오면 셋업 완료:

```
origin    https://github.com/내아이디/Zer0ne.git  # 내 복사본
upstream  https://github.com/cshbi/Zer0ne.git  # 원본
```

2부 과제할 때마다 반복하는 흐름

1 작업 전 — 원본 최신 받아오기 (꼭!)

다른 친구들 작업이 원본에 계속 쌓인다. 내 작업 전에 원본 최신을 당겨와야 뒤처지지 않고 충돌도 줄어든다.

```
git switch main
git fetch upstream      # 원본 최신 가져오기
git merge upstream/main # 내 main에 반영
```

웹이 편하면: 내 복사본 페이지의 **Sync fork** 버튼으로 맞춘 뒤 `git pull` 해도 된다.

2 작업용 브랜치 만들기

main에서 바로 하지 말고, 과제마다 새 갈래(브랜치)를 만들어 거기서 작업한다.

```
git switch -c feature/홍길동-2주차
```

이름 규칙: `feature/이름-작업내용` (예: `feature/kim-login`)

3 수정 → 담기 → 저장 → 올리기

```
# ... 파일 수정 ...
git add .
git commit -m "2주차 과제 제출"
git push -u origin feature/홍길동-2주차
```

안심 포인트: 여기까지는 내 복사본(origin)에만 올라간 것이다. **원본은 아직 안 바뀐다.** "합쳐주세요"라고 신청하는 건 다음 단계(PR)다.

4 Pull Request 보내기 — 웹에서

push 후 내 복사본 페이지에 가면 노란 띠로 **Compare & pull request** 버튼이 뜬다. 클릭 후 방향을 확인:

```
base:    cshbi/Zer0ne ← main          # 원본으로 (도착지)
compare: 내아이디/Zer0ne ← feature/홍길동-2주차 # 내 작업 (출발지)
```

제목·설명에 무엇을 했는지 적고 **Create pull request** 클릭 → 교수님께 신청서 도착. 끝!

3부 교수님이 승인한 다음 (재동기화)

내 PR이 승인(머지)되면 원본이 바뀐다. 다음 과제 전에 다시 최신을 맞추고, 끝난 브랜치는 정리한다.

```
git switch main
git fetch upstream
git merge upstream/main # 최신 원본 반영
git branch -d feature/홍길동-2주차 # 끝난 브랜치 삭제
# 다음 과제는 다시 2부-2 부터 (새 브랜치)
```

!! 자주 하는 실수 3가지

실수	왜 문제 / 바른 방법
원본을 clone함	원본엔 올릴 권한이 없어 push가 막힘 → 내 복사본 주소를 clone
동기화 안 하고 작업	옛날 버전 위에서 고쳐 충돌이 커짐 → 작업 전 항상 fetch upstream
main에서 바로 작업	PR이 지저분해짐 → 항상 switch -c 로 새 브랜치부터

⚠ 충돌(conflict)이 났을 때

두 사람이 같은 파일의 같은 줄을 다르게 고치면 충돌이 난다. **에러가 아니다**. Git이 "둘 중 뭘 남길지 사람이 골라"라고 멈춘 것뿐이다. 파일을 열면 이렇게 표시된다:

```
<<<<<<< HEAD
SCHOOL_NAME = "ZerOne Academy"      # 원본에 이미 있던 것
=====
SCHOOL_NAME = "제로원 코딩학원"      # 내가 보낸 것
>>>>>> 내브랜치
```

해결: 남길 한 줄만 남기고 <<< === >>> 표시줄을 모두 지운 뒤 다시 커밋한다.

```
git add 파일명
git commit -m "충돌 해결"
git push
```

우리 수업 작업 규칙

1단계(개인 과제): 각자 자기 폴더 안에서만 작업한다. 다른 사람 폴더·공통 파일은 건드리지 않는다. (충돌이 안 나는 구역)

```
ZerOne/
├── students/
│   ├── 01_홍길동/ ← 홍길동은 여기서만
│   ├── 02_김철수/ ← 김철수는 여기서만
│   └── ...
```

2단계(통합 과제): 공통 파일을 함께 수정한다. 이때 **충돌을 직접 경험**하게 된다. 당황하지 말고 위 "충돌 해결" 순서대로 처리. 작업 전 동기화(2부-1)를 빼먹지 말 것!

한 줄 요약 · 최초: Fork → clone(내 것) → remote add upstream | 매번: 동기화 → 브랜치 → add·commit·push → PR